So sánh các bộ phân lớp tự cài đặt trên bài toán nhận dạng chữ số viết tay MNIST

Gaussian Naïve Bayes, hồi quy softmax, MLP, AdaBoost-SAMME và một CNN nhỏ

Trần Trinh Nhân (24280088) Lâm Nhật Tiến (24280029)

Môn Nhận dạng, Khoa Toán – Tin học, Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM

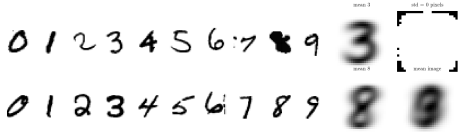
Đồ án giữa kỳ

Các giả thiết khác nhau về chữ số cho kết quả khác nhau đến mức nào?

Bài toán. Ảnh xám 28×28, 10 lớp; 60 000 ảnh huấn luyện và 10 000 ảnh test chính thức.

Câu hỏi nghiên cứu

- RQ1 So sánh bốn phương pháp dưới cùng một giao thức.
- RQ2 Độ phức tạp ảnh hưởng thế nào đến độ chính xác, chi phí và khả năng tổng quát hóa?
- RQ3 CNN khác MLP ở điểm nào?
- RQ4 Tăng cường dữ liệu ảnh hưởng thế nào đến độ bền?
- RQ5 Kết quả có nhất quán trên Fashion-MNIST và EMNIST Digits?



Mẫu ngẫu nhiên, ảnh trung bình lớp 3 và 8, các pixel có phương sai 0 (đen), ảnh trung bình toàn tập.

MNIST được chia 50 000/10 000/10 000; validation để chọn, test chỉ để báo cáo

- Chia phân tầng, seed 42, từ 60 000 ảnh huấn luyện; test chính thức giữ riêng.
- Chuẩn hóa bằng một cặp vô hướng $(\bar{x}, s) = (0.1306, 0.3081)$ tính *chỉ trên train*, áp lại cho validation, test và trình duyệt.
- Hàm tìm siêu tham số không nhận tập test.
- Tìm siêu tham số: quét từng tham số một (tự viết, không dùng Optuna); không xét tương tác.
- Độ đo: accuracy, macro precision/recall/F1, ma trận nhầm lẫn trên cả ba tập; so sánh cặp bằng McNemar chính xác.

Tự viết	**Thư viện**
Phần tính toán lõi, chỉ số, chia tập	scikit-learn, chỉ để đối chiếu
MLP, CNN: lan truyền tiến và ngược, SGD momentum, im2col	PyTorch chỉ làm tensor và phép nhân ma trận; không dùng autograd, lớp dựng sẵn hay bộ tối ưu

Naïve Bayes giả thiết pixel độc lập; softmax giả thiết biên quyết định tuyến tính

Gaussian Naïve Bayes

$$\hat{y} = \arg\max_k \left[\log \pi_k - \tfrac{1}{2} \sum_{j=1}^{784} \left(\log(2\pi\sigma_{kj}^2) + \frac{(x_j - \mu_{kj})^2}{\sigma_{kj}^2} \right) \right]$$

- Hệ số tương quan trung bình giữa hai pixel kề nhau trong cùng lớp là 0.65: giả thiết độc lập không đúng.
- Validation: 56.05% ở `var_smoothing`$=10^{-9}$, 79.67% ở 0.1; train và validation gần trùng nhau.

Hồi quy softmax, mini-batch gradient descent

$$p_k = \frac{e^{z_k}}{\sum_m e^{z_m}}, \quad z = W^\top x + b, \qquad \nabla_W L = \tfrac{1}{B} X^\top (P - Y) + \lambda W$$

- Accuracy trên chính tập train chỉ đạt 93.62%: mô hình bị giới hạn bởi dung lượng, không phải do overfitting.

MLP học biên phi tuyến bằng lan truyền ngược viết tay

$$z_l = a_{l-1} W_l + b_l, \quad a_l = \mathrm{ReLU}(z_l) \odot m_l$$

$$\delta_L = \tfrac{1}{B}(\mathrm{softmax}(z_L) - Y)$$

$$\delta_{l-1} = (\delta_l W_l^{\top}) \odot \mathbb{1}[a_{l-1} > 0] \odot m_{l-1}$$

$$\nabla_{W_l} = a_{l-1}^{\top} \delta_l, \quad v \leftarrow \mu v + g, \quad \theta \leftarrow \theta - \eta v$$

- Cấu hình chọn trên validation: hai lớp ẩn $(512, 256)$, $\eta = 0.05$, $\mu = 0.9$, batch 128; 535 818 tham số.
- Khởi tạo He, dropout inverted, dừng sớm: dừng ở epoch 14, khôi phục epoch 9.
- Kiểm thử: gradient khớp sai phân hữu hạn và autograd (chỉ dùng trong kiểm thử).

AdaBoost-SAMME cộng các cây sâu có trọng số: chính xác nhưng đắt

$$\alpha_t = \nu\Big(\ln\frac{1-\mathrm{err}_t}{\mathrm{err}_t} + \ln(K-1)\Big), \quad w_i \leftarrow \frac{w_i\, e^{\alpha_t \mathbb{1}[h_t(x_i)\neq y_i]}}{Z_t}, \quad H(x) = \arg\max_k \sum_t \alpha_t \mathbb{1}[h_t(x)=k]$$

- Bộ học yếu: cây CART có trọng số, tạp chất Gini, dựng bằng histogram 32 bin (tự viết bằng NumPy).
- Cấu hình: 393 cây, độ sâu 10, $\nu = 0.1$, mỗi cây xét một nửa số đặc trưng.
- Chi phí: 817.9 s huấn luyện, 18.705 ms mỗi ảnh (vòng lặp Python trên các cây), tệp xuất 9.2 MiB.
- Điểm số là tỉ lệ phiếu có trọng số, chưa hiệu chỉnh thành xác suất.
- Chưa có baseline một cây đơn, nên chưa định lượng được lợi ích riêng của việc kết hợp nhiều cây.

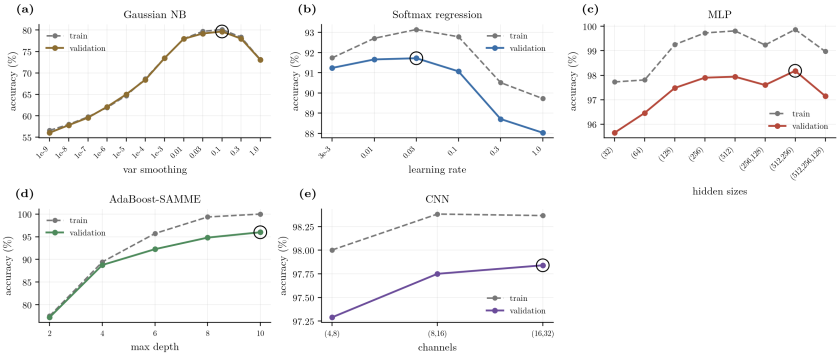
CNN và MLP dẫn đầu; thứ tự MLP–AdaBoost chưa chắc chắn

Mô hình	Train	Val	Test	$F1_{macro}$
Gaussian NB	80.13	79.67	81.32	81.16
Softmax	93.62	92.15	92.52	92.42
MLP	99.87	98.18	98.24	98.23
AdaBoost	100.00	97.59	97.60	97.58
CNN	99.40	98.65	98.80	98.79

Một seed (42); accuracy theo %. Khoảng tin cậy 95% của mỗi accuracy gần 98% rộng cỡ ±0.3 điểm.

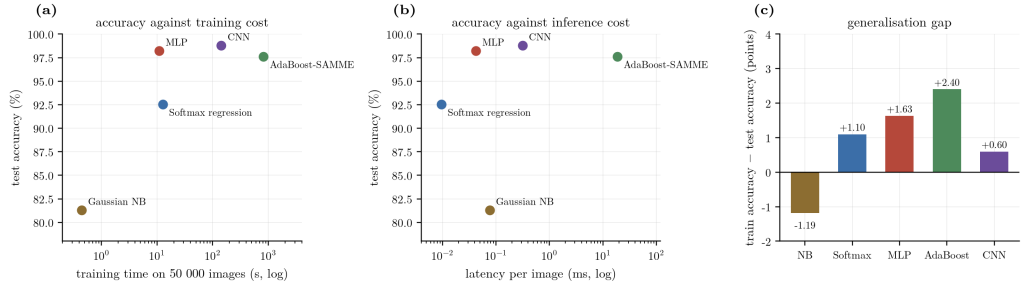
- McNemar: CNN–MLP 120 so với 64 ảnh ($p = 4.4 \times 10^{-5}$); MLP–AdaBoost 127 so với 63 ảnh ($p = 4 \times 10^{-6}$).
- McNemar chỉ phản ánh sai số lấy mẫu test, không phản ánh biến thiên theo seed.
- MLP, 3 seed: 98.24, 98.14, 97.69 (trung bình 98.02, độ lệch chuẩn 0.29); CNN, 2 seed: 98.80, 98.84.
- CNN trên MLP ổn định qua các seed; MLP trên AdaBoost thu từ 0.64 còn 0.42 điểm theo trung bình seed.
- Khớp scikit-learn (cùng siêu tham số): chênh 0.00–0.66 điểm, mỗi chênh lệch đã được giải thích trong báo cáo.

Mỗi mô hình phản ứng khác nhau với dung lượng và tốc độ học



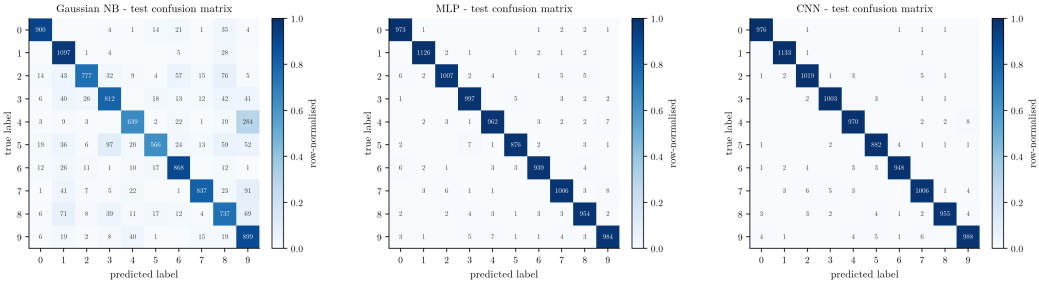
Vòng tròn: giá trị được chọn. AdaBoost (độ sâu 10) và CNN (kênh $(16, 32)$) đạt tốt nhất ở mép khoảng đã quét, nên có thể chưa được tinh chỉnh hết.

AdaBoost đắt nhất khi huấn luyện và suy luận



CPU, một máy; thời gian có thể chênh tới gấp đôi giữa các lần chạy (CNN: 143.0 s đến 312 s). Độ trễ của AdaBoost gồm cả chi phí vòng lặp Python trên 393 cây.

Các mô hình nhầm nhiều nhất ở những cặp có nét chung: $4 \to 9$, $7 \to 9$, $5 \to 3$



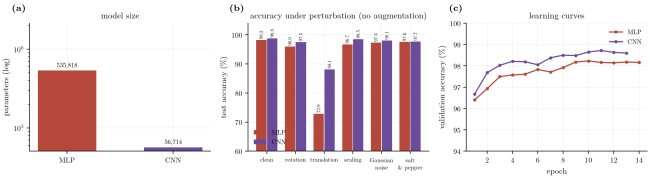
Lỗi trên test: Naïve Bayes 1868 (cặp $4 \to 9$: 284), MLP 176, CNN 120; softmax 748, AdaBoost 240.

CNN chính xác hơn MLP với ít tham số hơn 9.4 lần

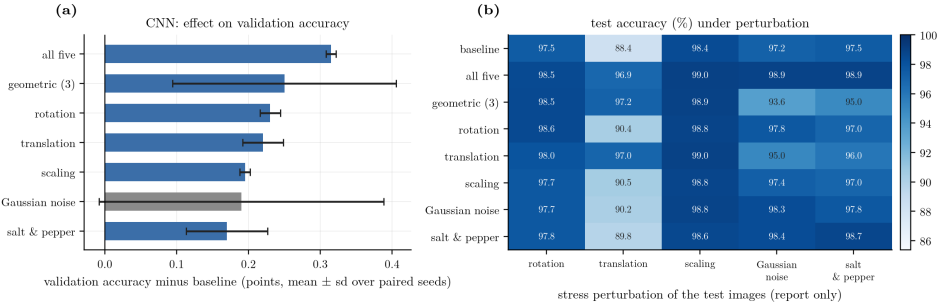
$1{\times}28{\times}28 \to 16{\times}26{\times}26 \to 16{\times}13{\times}13 \to 32{\times}11{\times}11 \to 32{\times}5{\times}5 \to 800 \to 64 \to 10$

56 714 tham số (MLP: 535 818); kích thước đọc từ `CNN.describe()`.

- Test: 98.80 so với 98.24%; train − test: 0.60 so với 1.63 điểm.
- Ảnh dịch 3 pixel: 88.14 so với 72.86%.
- Thời gian huấn luyện 143.0 s so với 11.0 s; 0.315 ms so với 0.042 ms mỗi ảnh.
- Chưa tách riêng vai trò của trọng số dùng chung và gộp cực đại.



Tăng cường dữ liệu tăng độ bền với đúng loại biến đổi



CNN, 2 seed. Mỗi điều kiện có 100 000 ảnh (gấp đôi điều kiện gốc), chọn theo validation (cả năm phép: test 99.25% so với 98.82%). Chỉ dịch hoặc nhóm hình học làm CNN kém bền hơn với nhiễu Gauss (97.2 → 95.0 và 93.6%). MLP, 3 seed: mức tăng validation (+0.09 đến +0.57) nằm trong độ lệch chuẩn ghép cặp (0.33–0.55); phép dịch nâng accuracy trên ảnh bị dịch từ 72.1 lên 94.5%.

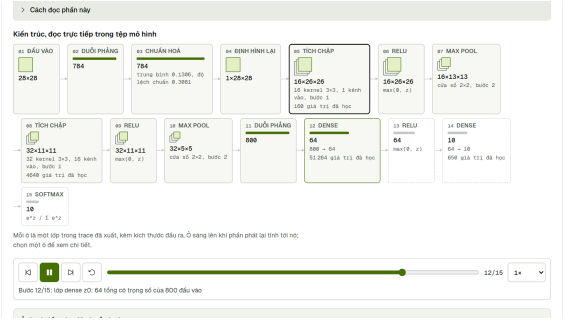
Thứ hạng giữ nguyên ở hai bộ chữ số, đổi ở Fashion-MNIST

Mô hình	MNIST	Fashion-MNIST	EMNIST Digits	MNIST → EMNIST
Gaussian NB	81.32	67.27	80.03	49.80
Softmax regression	92.52	84.27	93.81	53.59
MLP	98.24	88.59	98.36	76.70
AdaBoost-SAMME	97.60	84.74	97.38	73.69
CNN	98.80	88.49	98.64	92.15

Test accuracy (%). Mỗi bộ dữ liệu có phép chia, hằng số chuẩn hóa và siêu tham số riêng (quét rút gọn, một seed). Cột cuối: mô hình huấn luyện trên MNIST, đánh giá trên 40 000 ảnh test của EMNIST Digits; đây là các mô hình khác với cột EMNIST Digits. Fashion-MNIST gán 10 loại quần áo cho cùng các chỉ số 0–9 nên không có phép kiểm tra chéo hợp lệ với bộ chữ số. Trên Fashion-MNIST, MLP và CNN chênh 0.10 điểm, chưa đủ để kết luận.

Bản chạy thử vẽ lại phép tính của chính mô hình đã triển khai

- Năm mô hình, ba bộ dữ liệu, chạy trong trình duyệt (ONNX Runtime Web); mô hình tải từ Hugging Face theo revision ghim.
- Python sinh metadata (đồ thị lớp, hình dạng, tham số, mã băm); giao diện vẽ sơ đồ từ metadata và đối chiếu với đầu ra ONNX.
- Đây là trực quan hóa phép *tính*. Một bản đồ kích hoạt không chứng minh pixel hay bộ lọc đó gây ra quyết định.
- Điểm số của AdaBoost là tỉ lệ phiếu, không phải xác suất.



Kết luận và giới hạn

Điều có thể kết luận

- MNIST: CNN 98.80%, MLP 98.24%, AdaBoost 97.60%, softmax 92.52%, Naïve Bayes 81.32%.
- CNN hơn MLP ở mọi cặp seed, với ít tham số hơn 9.4 lần; thứ tự MLP–AdaBoost chưa chắc chắn.
- Tăng cường dữ liệu chủ yếu cải thiện độ bền với đúng loại biến đổi.
- Mô hình rời bộ dữ liệu huấn luyện mất nhiều độ chính xác, nhiều nhất ở softmax và Naïve Bayes.

Giới hạn

- Một seed cho mô hình chính.
- Quét từng tham số, một số giá trị tốt nhất ở mép khoảng quét.
- Tăng cường dữ liệu thiếu đối chứng cùng lượng dữ liệu.
- Thời gian đo trên một máy CPU.
- Chưa đo độ chính xác trên nét vẽ của người dùng.

Phân công: Trần Trinh Nhân 50%, Lâm Nhật Tiến 50%. Việc dùng trợ lý lập trình AI được công bố trong báo cáo (mục 9).

Mã nguồn: `https://github.com/TranTrinhNhan1/WriteRHand` (kho hiện ở chế độ riêng tư)
Demo: `https://trantrinhnhan1.github.io/WriteRHand/`
Mô hình: `https://huggingface.co/TranTrinhNhan1111/WriteRHand-MNIST`